MÜHENDİSLİK VE DOĞA BİLİMLER FAKÜLTESİ BİLGİSAYAR MÜHENDİSLİĞİ BÖLÜMÜ NESNEYE YÖNELİK PROGRAMLAMA DERSİ DÖNEM PROJESİ RAPORU

PROJE ADI: BANKA OTOMASYON SİSTEMİ

Hazırlayan: Sümeyye Şimşek- 24360859058

MAYIS 2026 BURSA

İÇİNDEKİLER

1. GİRİŞ

- 1.1. Projenin Amacı ve Kapsamı

2. TEKNİK DETAYLAR

- 2.1. Program Akışı ve Modüler Yapı
- 2.2. Sınıf Yapıları ve Kullanılan Veri Tipleri
- 2.3. Finansal İşlemlerin Hesaplama Mantığı
- 2.4. Girdi Doğrulama ve Hata Yönetimi
- 2.5. Test Verisi ve Başlangıç Konfigürasyonu (Data Seeding)

3. UML DİYAGRAMI VE SINIF İLİŞKİLERİ

4. EKSİKLİKLER VE GELİŞTİRMELER

- 4.1. Otomatik Raporlama ve Veri Dışa Aktarımı
- 4.2. Grafiksel Kullanıcı Arayüzü (GUI) Entegrasyonu

5. KARŞILAŞILAN SORUNLAR VE ÇÖZÜMLER

6. SONUÇ

7. KAYNAKÇA

1. GİRİŞ

1.1. Projenin Amacı ve Kapsamı

Bu proje, Bursa Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü, 1 Nesne Yönelimli Programlama dersi 2025-2026 Bahar Dönemi kapsamında 1. sınıf öğrencisi Sümeyye Şimşek tarafından dönem projesi olarak oluşturulmuştur.

Bu proje tamamen bireysel olarak geliştirilmiştir.

Bu projenin amacı, Java programlama dili kullanılarak konsol tabanlı bir Banka Otomasyon Sistemi geliştirmektir. Projenin ana hedefi, Nesne Yönelimli Programlama (Object-Oriented Programming - OOP) prensiplerini uygulayarak müşteri yönetimi, hesap işlemleri ve kredi kartı algoritmalarını dijital bir ortamda birleştirmektir. Program ilk başlatıldığında, kullanıcının interaktif bir menü üzerinden sistemi yönlendirmesini sağlayan ana arayüz ekranı karşılar **(Bkz. Şekil 1)**.

```
-- BANKA OTOMASYON SİSTEMİ --
1 - Yeni Müşteri Kaydı
2 - Müşteri Girişi / İşlem Yap
0 - Çıkış
Seçim:
```

Şekil 1:Banka Otomasyon Sistemi Ana Menü Arayüzü

Sistemin sınırları; yeni müşteri tanımlama, vadesiz ve yatırım hesapları oluşturma, hesaplar arası para transferi (havale), hesaba para yatırma ve kredi kartı borç/harcama yönetimini içermektedir. Kullanıcı menüden "1" numaralı seçeneği seçmesiyle beraber, sistem kişi bilgilerini (ad-soyad-iletişim numarası) talep ederek yeni müşteri kaydını oluşturuyor**.(Bkz. Şekil 2)**.

```
Seçim: 1

Ad Soyad: Sümeyye Şimşek
Email Adresi: simsek@gmail.com
Telefon Numarası: 05250250025
Müşteri sisteme kaydedildi: Sümeyye Şimşek
```

Şekil 2:Yeni Müşteri Kayıt Ekranı ve Veri Girdileri

2. TEKNİK DETAYLAR

2.1. Program Akışı ve Modüler Yapı

Yazılımın mimarisi, kodun okunabilirliğini ve sürdürülebilirliğini artırmak amacıyla modüler bir paket (package) yapısı üzerine kurulmuştur. Sistem işlevlerine göre ayrılmış sınıflardan oluşmaktadır:

- **com.bank.app.people:** `Kisi`, `Musteri` ve `BankaPersoneli` sınıfları ile aktörlerin kimlik yönetimini üstlenir.
- **com.bank.app.accounts:** `BankaHesabi` temel sınıfından türetilen `VadesizHesap` ve `YatirimHesabi` sınıflarını içerir.
- **com.bank.app.cards:** `KrediKarti` sınıfı ile limit ve borç işlemlerini ayrıştırır.
- **com.bank.app.main:** Uygulamanın ana döngüsünü barındıran `Main` sınıfı, kullanıcı etkileşimini ve senaryo testlerini yönetir. Program akışı, sistem kapatılana kadar aktif kalan sonsuz bir `while` döngüsü içerisinde çalışmaktadır.

2.2. Sınıf Yapıları ve Kullanılan Veri Tipleri

Veri bütünlüğünü sağlamak adına, sistemdeki değişken türleri kullanılan senaryolarara göre uygun olarak seçilmiştir. Müşterilerin telefon numaraları, başında "0" karakterini barındırdığı ve veri kapasitesi aşımını (overflow) önlemek gerektiği için `String` tipinde tanımlanmıştır. Finansal bakiye ve miktar işlemleri için ondalıklı hesaplamalara olanak tanıyan `double` veri tipi kullanılmıştır. Müşterilerin birden fazla hesaba ve kredi kartına sahip olabilmesi durumu ise, dinamik boyutlandırma sağlayan `ArrayList` yapıları ile kodlanmıştır. Tüm sınıf alanları `private` tanımlanarak Kapsülleme (Encapsulation) sağlanmış, verilere erişim `getter/setter` metotlarıyla sınırlandırılmıştır.

2.3. Finansal İşlemlerin Hesaplama Mantığı

Sistemdeki hesaplama mantığı, matematiksel kısıtlamalar baz alınarak tasarlanmıştır:

- **Para Transferi (Havale):** Gönderen hesabın bakiyesi kontrol edilir. Transfer miktarı bakiyeden küçük veya eşitse işlem gerçekleşir. Her iki hesabın işlem geçmişine (Transaction History) anlık olarak metin tabanlı kayıt atılır **(Bkz. Şekil 3)**.

```
-- TRANSFER BAŞARILI --
Gönderen IBAN      : TR736830247669269003002929
Alıcı IBAN         : TR259011592150021910283213
İşlem Öncesi Bakiye: 150.0 TL
Transfer Edilen    : 100.0 TL
Kalan Güncel Bakiye: 50.0 TL
```

Şekil 3: Hesaplar Arası Para Transferi İşlem Çıktısı

- **Kredi Kartı Algoritması:** Yeni bir kredi kartı oluşturulduğunda varsayılan olarak 50.000 TL limit atanmaktadır. Harcama yapıldığında sistem kullanılabilir limiti düşürürken güncel borcu artırır. Borç ödeme işlemi sadece müşterinin Vadesiz Hesap nesnesi üzerinden gerçekleştirilebilir.

2.4. Girdi Doğrulama ve Hata Yönetimi

Uygulamanın çalışma sürekliliğini sağlamak için konsol girdileri üzerinde katı doğrulama (validation) süreçleri kodlanmıştır.

- Kullanıcıdan sayı beklenen alanlarda (menü seçimi, miktar girişi) harf girilmesi durumunda programın çökmesini (InputMismatchException) engellemek için `sayiAl()` ve `miktarAl()` isimli özel metotlar yazılmıştır. Bu metotlar, string içerisindeki karakterleri `Character.isDigit()` ile kontrol ederek geçersiz girişlerde kullanıcıyı tekrardan veri girişine yönlendirir **(Bkz. Şekil 4).**

```
> Müşteri: Sümeyye Şimşek
1 - Hesap Aç (Vadesiz/Yatırım)
2 - Para Yatır
3 - Para Transferi
4 - Kredi Kartı İşlemleri
5 - Hesap Kapat (Sil)
6 - İşlem Geçmişini Gör
0 - Ana Menü
İşlem: a
HATA! Menüde olmayan bir sayı girdiniz. Lütfen 0 ile 6 arasında bir seçim yapınız!
```

Şekil 4: Hatalı Veri Girişinde Sistemin Hata Yönetimi Döngüsü

- Tip uyuşmazlıklarını önlemek için `instanceof` operatörü kullanılmıştır. Yatırım hesabından havale yapılmaya çalışıldığında ortaya çıkabilecek `ClassCastException` (Sınıf Dönüştürme Hatası) engellenmiştir.

2.5. Test Verisi ve Başlangıç Konfigürasyonu (Data Seeding)

Sistemin geliştirme ve test aşamalarında, programın her çalıştırılışında manuel veri girişini önlemek amacıyla Main sınıfı içerisinde varsayılan bir müşteri kaydı oluşturulmuştur. Sistem başlatıldığı anda oluşturulan müşteri nesnesine ait vadesiz hesaba 5000 TL başlangıç bakiyesi tanımlanmaktadır **(Bkz. Şekil 5)**.

```
Başarılı! Vadesiz hesap eklendi. IBAN: TR259011592150021910283213
Varsayılan müşteri Yaren Yoldaş için 5000.0 TL başlangıç bakiyesi başarıyla tanımlandı.
Müşteri sisteme kaydedildi: Yaren Yoldaş
```

Şekil 5: Sistemin Varsayılan Müşteri Ataması

Bu test parametresi; para transferi ve kredi kartı oluşturma süreçlerinin beklemeye gerek kalmadan doğrudan test edilmesini sağlamıştır.

3. UML DİYAGRAMI VE SINIF İLİŞKİLERİ

Banka Otomasyon Sistemi'nin tasarımı, Nesne Yönelimli Programlama (OOP) ilkelerine tam uyum sağlayacak şekilde hiyerarşik bir yapıda tasarlanmıştır. Sistemin mimari şeması ve sınıflar arasındaki etkileşim ağını gösteren UML diyagramı ayrıntılı olarak sunulmuştur **(Bkz. Şekil 6)**.

3.1. Kalıtım (Inheritance) ve Hiyerarşik Yapı

Sistemde kod tekrarını önlemek ve mantıksal bir düzen kurmak amacıyla iki ana hiyerarşi oluşturulmuştur:

- **Kişi Hiyerarşisi:** `Kisi` sınıfı; ad, soyad ve iletişim bilgilerini tutan temel (base) sınıftır. `Musteri` ve `BankaPersoneli` sınıfları bu sınıftan türetilerek "is-a" (bir kişidir) ilişkisini doğrulamaktadır.
- **Hesap Hiyerarşisi:** `BankaHesabi` sınıfı, tüm hesap türlerinde ortak olan IBAN ve bakiye gibi özellikleri tanımlar. `VadesizHesap` ve `YatirimHesabi` sınıfları bu sınıftan miras alarak kendilerine özgü fonksiyonlar (para transferi, yatırım yönetimi vb.) oluşturulmuştur.

3.2. Bileşim (Composition) ve Sınıf İlişkileri

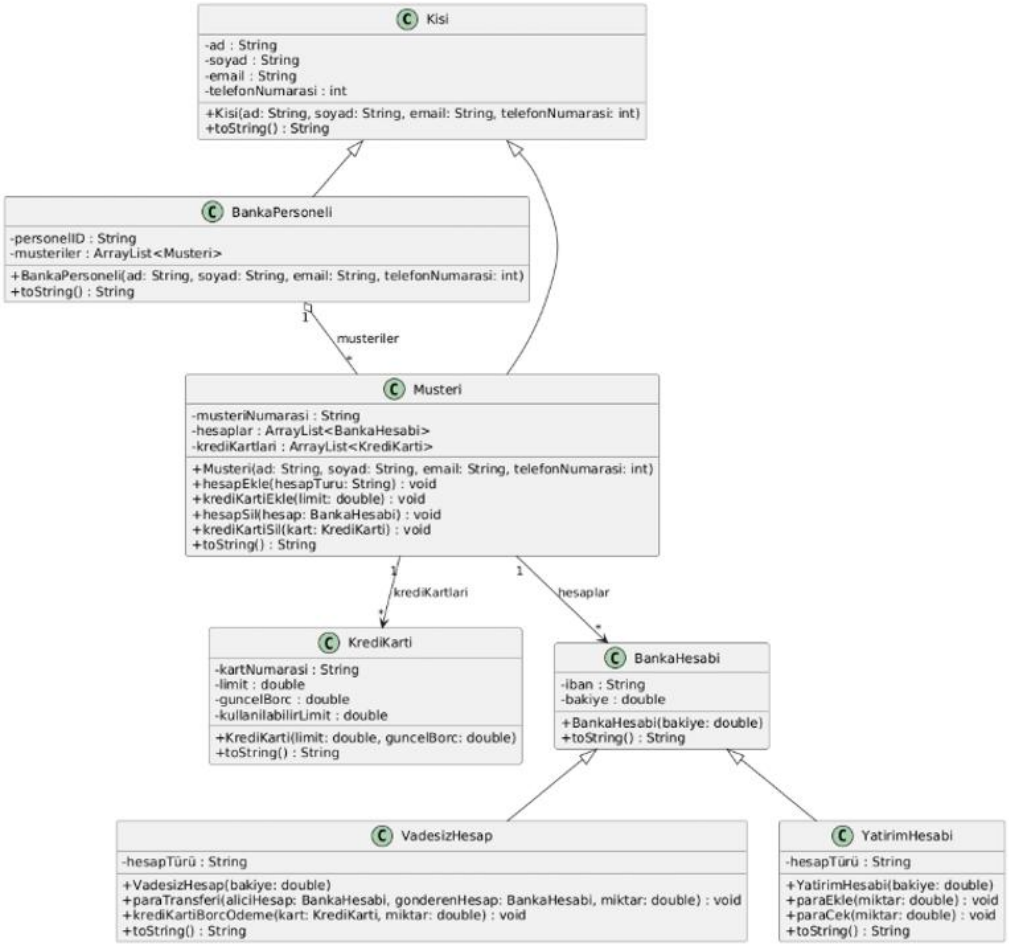
Sistemdeki sınıflar sadece kalıtım yoluyla değil, aynı zamanda sahip olma (has-a) ilişkisiyle de birbirine bağlıdır:

- **Müşteri Nesnesi:** Her bir `Musteri` nesnesi, bünyesinde `ArrayList` yapısı içerisinde birden fazla `BankaHesabi` ve `KrediKarti` nesnesini barındırmaktadır.
- **Servis Katmanı:** `BankaService` sınıfı, sistemdeki tüm `Musteri` nesnelerini merkezi bir listede birleştirerek verilerin tek bir noktadan yönetilmesini (kaydetme, listeleme) sağlar.

3.3. Kapsülleme (Encapsulation) Uygulaması Diyagramda görülen tüm eksi (-) işaretli alanlar `private` erişim belirleyicisini temsil etmektedir. Bu sayede veriler dış müdahalelere karşı korunmuş, sınıflar arasındaki veri akışı sadece artı (+) işaretli `public` metotlar (getter/setter) aracılığıyla kontrol edilebilir hale getirilmiştir.

3.4. Çok Biçimlilik (Polymorphism) Uygulaması

Sistemde çok biçimlilik, `BankaHesabi` üst sınıfı referans alınarak alt sınıfların (`VadesizHesap` ve `YatirimHesabi`) ortak bir liste yapısında işlenmiştir. Müşterinin sahip olduğu farklı hesap türleri, tek bir `ArrayList<BankaHesabi>` koleksiyonu içerisinde saklanmaktadır. Çalışma zamanında (runtime) bu hesaplar üzerinde işlem yapılırken (örneğin kredi kartı borç ödeme kontrolünde), `instanceof` operatörü kullanılarak nesnenin bellekteki dinamik türü tespit edilmekte ve işlemlerin yalnızca geçerli sınıf kapsamı dahilinde yürütülmesi sağlanmaktadır."



Şekil 6: Banka Otomasyon Sistemi UML Sınıf Diyagramı

4. EKSİKLİKLER VE GELİŞTİRMELER

Geliştirilen Banka Otomasyon Sistemi, nesne yönelimli programlama standartlarına uygun ve hatasız bir şekilde çalışmaktadır. Bununla birlikte, yazılımın endüstriyel standartlarda bir ürüne dönüşebilmesi için gelecekte planlanan geliştirme alanları aşağıda maddeler halinde sunulmuştur:

4.1. Şifreli Giriş Mekanizması

Mevcut sistemde müşteri numarası ile doğrudan giriş yapılabilmektedir. Sistemin güvenliğini artırmak amacıyla, her müşteri için kayıt aşamasında belirlenecek 4 haneli bir PIN kodu veya şifre sorgulama ekranı eklenebilir. Bu sayede kullanıcıların hesaplarına yetkisiz erişimlerin engellenmesi simüle edilebilir.

4.2. Dosya Kayıt ve Okuma Sistemi (File I/O)

Program şu anda verileri sadece çalışma zamanında (bellekte) tutmaktadır. Basit bir dosya işleme yapısı kullanılarak; müşteri bilgileri, hesap bakiyeleri ve işlem geçmişi bir `.txt` veya `.csv` dosyasına kaydedilebilir. Böylece program kapatılıp tekrar açıldığında eski verilerin otomatik olarak yüklenmesi sağlanabilir ve daha kalıcı bir sistem oluşturulabilir.

4.3. Çoklu Dil Desteği (Localization)

Kullanıcı deneyimini geliştirmek amacıyla programa dil seçeneği eklenebilir. Başlangıçta sorulacak bir soru ile tüm menülerin ve hata mesajlarının hem Türkçe hem de İngilizce olarak görüntülenmesi sağlanarak yazılımın evrensel standartlara uygunluğu artırılabilir.

5. KARŞILAŞILAN SORUNLAR

Bu proje ile Java programlama dili ve Nesne Yönelimli Programlama mantığı kullanılarak işlevsel bir banka yazılımı üretilmiştir. Geliştirme sürecinde karşılaşılan temel teknik sorunlar ve bu sorunlara yönelik geliştirilen çözümler aşağıda özetlenmiştir:

- **Veri Tipi Kaynaklı Çökmeler:** Kullanıcıdan sayı beklenen alanlara harf veya geçersiz karakter girildiğinde programın InputMismatchException hatası vererek durması en büyük teknik sorundu. Bu durum, girdi verilerini doğrudan sayı olarak almak yerine String olarak alan ve her bir karakteri döngü içerisinde denetleyen miktarAl() ve sayiAl() metotlarının yazılmasıyla çözülmüştür.
- **Hatalı İşlem Mantığı:** Para transferi veya kredi kartı borç ödeme işlemlerinde, hesabın mevcut bakiyesinden daha yüksek tutarların girilmesi bakiyenin eksiye düşmesine neden oluyordu. Bu mantıksal hatayı engellemek için her işlem öncesine katı if-else kontrol blokları eklenerek bakiye yetersizliği durumunda kullanıcının uyarılması sağlanmıştır.
- **Liste ve Arama Hataları:** Sistemde kayıtlı olmayan bir müşteri numarası veya IBAN girildiğinde arama algoritmalarının null nesne döndürmesi sorun yaratmıştır. Bu sorun, her arama işleminin ardından bir null kontrolü yapılması ve nesne bulunamadığında işlemin güvenli bir şekilde iptal edilip menüye dönülmesiyle aşılmıştır.

6. SONUÇ

Bursa Teknik Üniversitesi Bilgisayar Mühendisliği Bölümü bünyesinde yürütülen bu proje kapsamında, Nesne Yönelimli Programlama (OOP) prensiplerine dayalı, fonksiyonel bir Banka Otomasyon Sistemi geliştirilmiştir. Yazılımın geliştirilme sürecinde Java dilinin sunduğu kalıtım, çok biçimlilik, kapsülleme ve soyutlama yetenekleri etkin bir şekilde kullanılmıştır.

Sistem; yeni müşteri kaydı, hesap yönetimi, para transferleri ve kredi kartı algoritmalarını veri bütünlüğünü koruyarak gerçekleştirebilmektedir. Özellikle girdi doğrulama mekanizmaları sayesinde kullanıcı hataları minimize edilmiş ve uygulamanın kararlılığı ve güvenliği sağlanmıştır. Proje, bir bankacılık yazılımının temel iş mantığını başarıyla simüle etmiş olup, gelecekte veritabanı ve grafik arayüz entegrasyonu ile daha profesyonel bir yapıya dönüştürülmeye hazırdır.

7. KAYNAKÇA

- Liang, Y. D. (2018). *Introduction to Java Programming and Data Structures, Comprehensive Version*. Pearson Education.
- Oracle Corporation. (2026). *Java Platform, Standard Edition Documentation (JDK 21)*. docs.oracle.com.
- Bursa Teknik Üniversitesi, BML0121 Nesne Yönelimli Programlama Ders Notları (2026).
- Deitel, P., & Deitel, H. (2017). *Java How to Program, Early Objects*. Pearson.

Github Repo: GitHub Linki

Youtube: Proje Tanıtım Videosu